MUXQ: Mixed-to-Uniform precision matriX Quantization via Low-Rank Outlier Decomposition

이승섭, 김인서, 김선욱
고려대학교 전기전자공학부
e-mail: {tmdtjq789, kiminse123, seon}@korea.ac.kr

MUXQ: 저랭크 Outlier 분해를 활용한 혼합 정밀도 행렬의 균일 정밀도 양자화

Seoung Sub Lee, In Seo Kim, and Seon Wook Kim
School of Electrical Engineering
Korea University, Seoul, Korea

Abstract

대규모 언어모델(LLM)은 다양한 자연어 처리 과제에서 뛰어난 성능을 보이지만, 막대한 파라미터 수로 인해 메모리와 연산 자원 소모가 심각하다. 특히 NPU 기반 온디바이스(On-device) 환경에서는 FP16/FP32 연산이 비효율적이므로, 정수(INT) 연산 기반의 양자화(Quantization)가 필수적이다. 그러나, 기존 방법(ZeroQuant, LLM.int8(), SmoothQuant 등)은 활성값(Input Activation) outlier 문제와 하드웨어 비효율을 완전히 해결하지 못했다.

본 연구에서는 이러한 한계를 극복하기 위해 MUXQ(Mixed-to-Uniform Quantization)를 제안한다. MUXQ는 활성값 채널의 outlier 채널을 탐지하고, 해당 채널을 분산시키는 작은 보조 행렬(small auxiliary matrix)을 추가하여 outlier의 magnitude를 완화한다. 이를 통해 활성값의 outlier또한 낮은 정밀도의 INT 수준으로 양자화를 가능하게 하며, 연산에 있어서 하드웨어 친화적인 구조를 유지한다. GPT2계열(small-0.1B, medium-0.3B, large-0.7B) 모델과 WikiText-2 데이터셋을 이용한 실험 결과, MUXQ는 기존의 naive quantization 대비 더 낮은 perplexity를 유지하며, per-tensor quantization에서는 FP16수준의 정확도로 활성값과 가중치(Weight)를 각각 INT8로 양자화 한다. MUXQ는 소규모의 추가 연산으로도 안정적인 저정밀도 추론을 가능하게 하며, 다른 양자화 기법과 병행 적용이 가능한 확장성을 가진다. 따라서 MUXQ는 엣지 디바이스 환경에서 효율적이고 정확한 LLM 추론을 위한 새로운 방향을 제시한다.

I. Introduction

최근 대규모 언어모델(Large Language Model, LLM)은 자연어 처리뿐 아니라 질의응답, 요약 등 다양한 응용분야에서 탁월한 성능을 보여주고 있다. LLM 수요가 증가함에 따라 사용자들의 빠르고 정확한 모델에 대한 요구 또한 증가했다. Transformer 기반의 LLM 모델들은 정확도를 위해서 수십억개에 달하는 파라미터를 학습 및 추론을 해야 하지만, 실시간 추론을 제공하기 위해서는 파라미터 수를 계속해서 증가시킬 수 없다[1]. 또한 이러한 파라미터 수의 증가는 GPU(Graphic Processing Unit)과 같은 하드웨어 가속기의 메모리 사용량을 점차 늘려 나갔으며[2], 이에 따라 온디바이스 환경에서는 실시간 추론이 더욱 어려워졌다.

최근 스마트폰, 임베디드 시스템 등 엣지 디바이스에서 AI 활용이 급격히 확대되고 있으며, 이에따라 기존에 부동 소수점 연산(FP16, FP32)으로 실행하던 LLM 모델을 INT 연산을 할 수 있도록 경량화하고 효율적으로 추론하는 기술의 중요성이 커지고 있다[3]. 온디바이스에서 AI 가속을 지원하는 NPU(Neural Processing Unit)의 경우는 특히

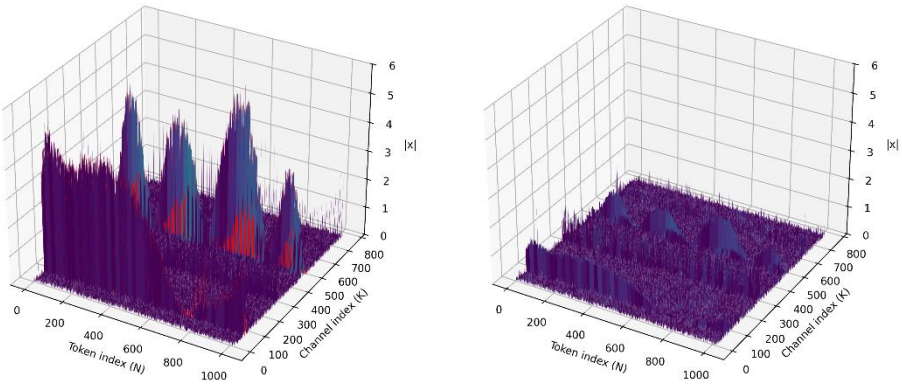

그림 1. 좌)활성값 outlier의 분포가 특정 channel에 집중되어 있는 모습. 우)MUXQ 이후 outlier channel들의 magnitude가 작아진 모습.

INT 연산에 최적화 되어있어, 이러한 환경에서 FP 연산을 그대로 구현하는 것은 비효율적일 뿐만 아니라, 추론에 있어 심각한 제약을 초래한다.

메모리 병목 또한 현재 LLM 구조의 주요 병목으로 주목받고 있다. KV-cache 의 경우 모델 크기가 커질수록 GPU 메모리의 절반 이상을 점유하기도 하며, 이는 모델 사용을 어렵게 만든다[2]. 따라서 저정밀 정수(INT4, INT8) 연산으로 모델을 압축하는 양자화(Quantization) 기술로 FP 연산을 대신하고, 메모리 병목을 해결해야 하는 것이 현재 상황이다.

양자화는 모델 가중치와 활성값을 낮은 정밀도 표현으로 변환하여 메모리 사용량을 줄이고 연산속도를 높이는 방법이다[4]. 예를 들어, FP16 에서 INT8 로 양자화할 경우, 메모리 사용량을 절반으로 줄이면서 이론적으로는 행렬 곱 연산(GEMM)을 두 배 이상 가속시킬 수 있다. 그러나, 활성값 행렬에서 나타나는 outlier 는 이러한 양자화를 어렵게 한다. 그림 1 은 활성값의 outlier 가 특정 몇몇의 채널만을 따라 형성된 모습을 보여준다. 이를 구조적으로 제거하거나 무시할 경우 모델의 정확도가 급격하게 떨어진다[5].

이러한 문제를 해결하기 위해 LLM.int8()[6], SmoothQuant[5] 등 다양한 양자화 기법 연구들이 등장하였다. LLM.int8()은 outlier feature 를 분리하여 FP16 으로 연산하는 mixed-precision decomposition 을 제안함으로써 정확도 저하를 방지했으나, 포함된 일부 FP16 연산이 하드웨어에서 메모리 접근 불규칙성(irregular memory access)을 발생시켜 하드웨어 효율성이 떨어지며 mixed-precision 으로 인한 연산 효율 감소 또한 존재한다. 한편, SmoothQuant 는 활성값의 scaling factor 를 가중치로 migrate 함으로써 양자화 난이도를 줄였지만, 활성값의 outlier 가 다른 원소들에 비해 여전히 크다는 점을 해결하진 못했다.

이처럼 기존 연구들은 정확도 손실을 최소화하기 위해 mixed-precision 또는 scaling-based 완화법을 적용했지만, 하드웨어 효율을 지키지 못하거나 활성값의 outlier 가 여전히 존재한다. 따라서 LLM 추론에서는 이미 저정밀 양자화가 가능한 가중치보다 활성값의 outlier 의 탐지 및 처리(handling)하는 것이 성능 보존의 핵심 과제이다.

본 연구에서는 이러한 문제를 근본적으로 해결하기 위해 MUXQ(Mixed-to-Uniform precision matrix Quantization)를 제안한다. MUXQ 는 기존의 mixed-precision 양자화 방식(FP16+INT8)이 갖는 하드웨어 병목을 제거하고, 균일하게 INT8 저정밀 양자화를 수행하는 새로운 양자화 구조이다. 구체적으로, MUXQ 는 활성값의 outlier 를 탐지해 각 layer 의 입력 활성 행렬(input activation matrix)에 작은 보조 행렬(small auxiliary matrix)을 추가한다. 이 보조 행렬은 입력 활성값 행렬에서 나타나는 outlier 의 magnitude 를 split 해 나눠 갖도록 한다.

따라서 보조행렬이 본래 활성값 행렬의 outlier 를 완화시켜 저정밀 양자화를 가능하도록 하며, 결과적으로 활성값이 균형 잡힌 분포를 갖게 되어 양자화 난이도가 완화되고 INT 정밀도만으로도 높은 정확도를 유지할 수 있다. 그림 1 의 오른쪽 그림은 MUXQ 를 통해 활성값이 더욱 균형 있는 분포를 갖는 모습이다.

MUXQ 는 모든 연산은 INT 기반으로 수행되므로 NPU 와 같은 하드웨어 친화적으로 동작하며 메모리 접근 불규칙성 문제가 사라진다. MUXQ 는 소규모의 추가 메모리 사용량과 연산량 증가를 수반하지만, 대신 더 낮은 비트 정밀도(INT8 또는 INT4)에서의 안정적인 추론이 가능하며, 전체 시스템

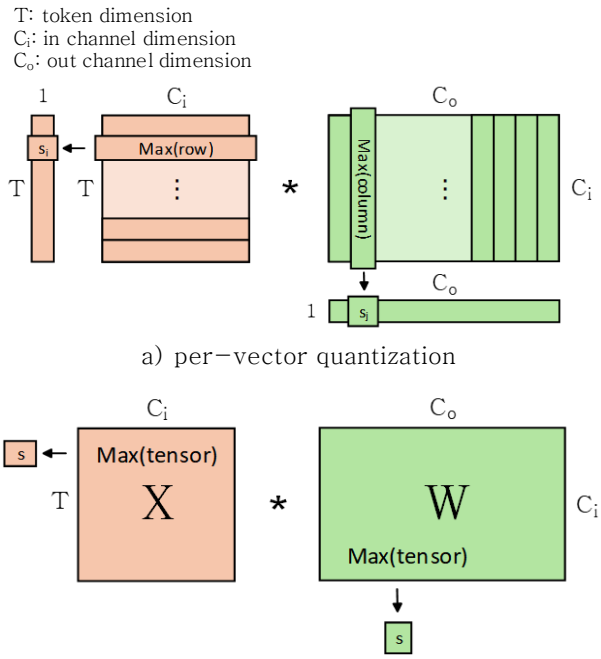

그림 2. 각각 a) per-vector, b) per-tensor 양자화 과정. Per-vector의 경우, 활성값과 가중치가 각각 per-row 또는 per-channel quantization이 되는 모습으로, 해당 벡터의 최대값으로 s_i(scaling factor)가 결정된다.

응답지연시간과 메모리 사용량을 동시에 줄일 수 있다. 본 실험에서는 MUXQ 는 0.1B 수준인 GPT2 모델에서 mixed-precision 으로 계산한 LLM.int8()과 비슷한 정확도를 보이며 per-tensor INT8 로 통일된 양자화에 성공했다.

본 논문의 기여는 저정밀 양자화를 1) 활성값 행렬에 새로운 보조 행렬을 추가해 구현해낸 점과, 2) 다른 양자화 기법과 함께 병합되어 사용될 수 있다는 점이다. 2)의 예를 들면, 현재 MUXQ 에서 생성된 활성값 행렬에도 SmoothQuant 의 양자화 난이도 이관 방식을 적용해 더욱 성능향상을 도모할 수 있다.

본 논문의 나머지 부분은 다음과 같이 구성된다. ‘Ⅱ. 배경’에서 양자화와 양자화에 영향을 주는 outlier 에 대해서 서술한다. ‘Ⅲ. MUXQ 구조’에서는 outlier 의 영향을 어떻게 MUXQ 에서 줄일 수 있는지 확인한다. ‘Ⅳ. 성능분석’에서는 양자화 난이도를 다르게 하며, MUXQ 와 naive quantization, LLM.int8()의 성능을 GPT2 계열 모델에서 비교한다. ‘V. 결론’에서는 MUXQ 의 의의와 발전가능성에 대한 이야기로 마무리한다.

Ⅱ. 배경

2.1 양자화

양자화는 FP로 표현된 값을 INT수준으로 매핑하는 과정을 의미한다. 대부분의 하드웨어 연산에서는 FP 연산보다는 INT 연산이 더 효율적이기에 양자화 이후 INT 연산을 한 이후 다시 복원하는 방식으로 하드웨어의 효율을 높일 수 있다[7].

기본적인 양자화 방법으로는 abs max quantization이 있다. 이는 양자화 대상 단위내에서 절댓값이 가장 큰 원소를 기준으로 양자화 하는 방식이다. 아래 수식은 abs max quantization을 나타낸다. 양자화 범위 내의 최댓값을 scaling factor로 나누고 반올림해 정수를 만든다. 정수 행렬곱 이후 다시 scaling factor를 곱해 복원하는 방식이다.

$$\bar{X}_{INT8} = \text{round}\left(\frac{X_{FP16}}{s_X}\right), s_X = \frac{\max(|X_{FP16}|)}{2^7 - 1} \quad (1)$$

$$\bar{W}_{INT8} = \text{round}\left(\frac{W_{FP16}}{s_W}\right), s_W = \frac{\max(|W_{FP16}|)}{2^7 - 1} \quad (2)$$

$$\bar{Y}_{FP16} = s_X \cdot s_W \cdot (\bar{X}_{INT8} \cdot \bar{W}_{INT8}) \quad (3)$$

양자화 대상 단위는 scaling factor를 공유하는 범위에 따라 granularity(세분화 수준)를 결정한다. granularity는 per-group, per-vector, per-tensor로 나뉜다. 이때 per-vector 방식은 양자화 대상으로 행렬의 token vector혹은 channel vector를 선택함에 따라 각각 per-token, per-channel 방식으로 나뉜다. per-group은 양자화 오버헤드가 너무 크기에[8] 이번 실험에서는 per-vector, per-tensor의 granularity로 실험해보았다. 그림 2는 granularity에 따른 양자화 방식을 나타낸 것이다.

2.2 Outlier Handling

Outlier의 magnitude는 타 원소 값들에 비해 비정상적으로 높아 scaling factor를 매우 작게 만든다. 그림 3은 outlier에 의해 각 원소들이 고르게 분포하지 못하는 모습을 설명한 그림이다. 따라서 이 outlier channel의 magnitude만을 다른 일반적인 채널과 비슷한 수준으로 낮출 수 있다면 정확도를 유지하면서 저정밀의 per-vector, per-tensor quantization을 가능하게 한다. 앞선 연구들(그림 1)을 통해 outlier feature는 channel-wise하게 나타남을 알 수 있었다. 따라서

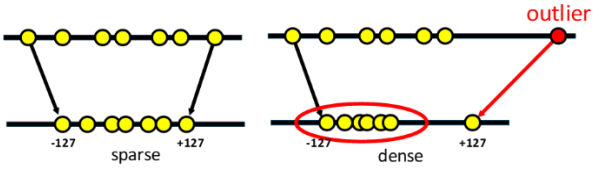

그림 3. outlier가 scaling factor에 영향을 주며 분포가 dense해진 모습. 이는 quantization error를 증가시킨다.

활성값을 per-channel로 양자화한다면 정확도가 훨씬 높아질 것을 예상할 수 있다[9]. 그러나, per-channel 활성값 양자화는 하드웨어 가속기의 GEMM 커널에서는 이용되기 어렵기 때문에, 이전 연구들은 대부분 활성값 양자화를 per-token 혹은 per-tensor를 통해 진행했다[5][6]. 본 연구 또한 이를 고려해 활성값을 per-token과 per-tensor 양자화로 구현하였다.

Ⅲ. MUXQ 구조

앞서 언급한 outlier 의 channel-wise 한 특징과 너무 큰 magnitude 를 갖는다는 특징을 보아 활성값 outlier channel 의 magnitude 를 줄이는 것이 해결책 중 하나라는 것을 알 수 있다. 따라서 본 연구는 활성값 행렬에 새로운 행렬을 추가하는 방법으로 outlier channel magnitude 를 분산시켜 per-tensor 수준에서 양자화를 가능하게 하는 MUXQ(Mixed to Uniform precision matriX Quantization)를 제시한다.

3.1 연구동기

LLM fine-tuning 시 가중치에 Low Rank matrix 를 추가하여 해당 행렬만을 학습시키는 방법이 존재한다[10]. 해당 방법은 fine-tuning 학습 시간을 효과적으로 감소시키며 현재까지도 fine-tuning 시 널리 쓰이는 기법이다. 이에 이용하여 MUXQ 는 가중치가 아닌 활성값에 Low Rank Matrix 를 추가해 활성값 outlier channel magnitude 를 나눠 가진다면 저정밀의 양자화를 가능하게할 수 있을 것이라는 아이디어에서 고안되었다.

3.2 Outlier 값 감소

FP 숫자형식은 sign, exponent, mantissa 로 구성된다. 이 중, exponent 는 magnitude 에 큰 영향을 미친다. 따라서 exponent 를 적절하게 shift 함으로 써 다른 채널과 비슷한 수준의 magnitude 를 갖는다면, naive quantization 방식에 대비해서 양자화 오류가 크게 감소할 것이다. 그러나, outlier channel exponent 를 보정하기 위해서는 행렬곱 연산 직후에 과정이 이루어져야 한다. 이는 systolic array, MAC 연산기와 같은 하드웨어에서는 실행되기 어렵다. 따라서 해당 outlier channel 들의 결과를 다시 shift 보정하는 것보다, Low Rank Matrix 를 추가해 따로 계산한 값을 더해 보정하는 방법을 이용해 하드웨어 효율을 올릴 수 있다.

3.3 Outlier decomposition 방법

본 연구에서는 MUXQ 의 Low Rank decomposition 이후 새롭게 형성된 기존 행렬과 Low Rank matrix 행렬을 각각 Body 와 Auxiliary 라 정의한다.

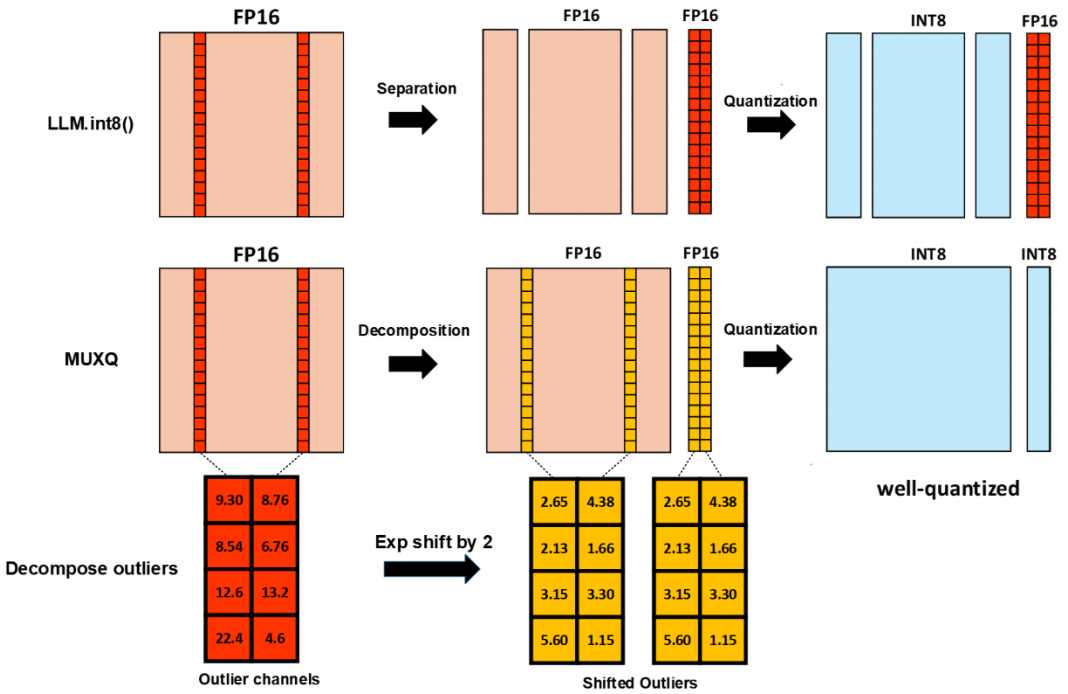

그림 5. MUXQ의 구조와 LLM.int8()의 구조를 비교한 그림.
그림 하단은 MUXQ의 α, β가 각각 2인 경우를 가정해 outlier를 decompose하는 예시를 보여준다.

$$X_{outlier} = \frac{1}{\alpha}Body_{outlier} + \frac{1}{\beta}Aux \qquad (4)$$

$$Y_{total} = Y_{body} + (\alpha - 1 + \beta)Y_{AUX} \qquad (5)$$

MUXQ 구조에서는 해당 outlier 의 exponent 를 얼마나 shift 할지 결정하는 α, β를 고르는 것이 중요한 문제 중 하나가 될 것이다. MUXQ 는 outlier 의 선정 기준을 LLM.int8()에서 참고 했음을 앞서 밝혔다. 해당 연구에서의 outlier channel 정의 기준은 'magnitude 가 6 이상인 원소를 포함하는 channel'이므로 활성값을 다른 채널들의 평균 수준으로 내리기 위해서는 α=β=2 로 선정하였다. 그러나, 아래 수식을 보면 알 수 있듯이, α = 1, β =1 일 때는 MUXQ 의 구현을 두 행렬곱 결과를 단순히 더함으로써 구현할 수 있지만, 두 값이 1 이 아닌 경우에는 추가적인 오버헤드가 발생할 수 있기에 이를 고려해서 α 와 β 를 설정해야 한다.

$$Y_{body} = \frac{1}{\alpha}Body \cdot W;\ Y_{AUX} = \frac{1}{\beta}Aux \cdot W \qquad (6)$$

$$Y_{total} = Y_{body} + (\alpha - 1 + \beta)Y_{AUX} \qquad (7)$$

Ⅳ. 성능분석

4.1 실험 환경 구성

Evaluation method. 본 절에서는 제안한 MUXQ의 성능을 검증하기 위한 실험 환경을 구성하였다. 비교 대상으로는 naive quantization 과 MUXQ 그리고 LLM.int8()을 함께 평가한다. 모델의 양자화 난이도(quantization difficulty)를 단계적으로 조정하면서 각각의 quantization method가 보이는 성능 변화를 비교하였다. 또한 성능 평가는 언어 모델링 perplexity를 기반으로 진행하였다.

Model and dataset. 평가 데이터셋으로는 wikitext-2[11] 데이터셋을 사용하였다. 해당 데이터셋은 wikipedia 기반의 자연어 문장으로 구성된 공개 언어 모델 학습용 벤치마크로, 다양한 모델의 언어 생성 능력을 비교하기에 적합하다. 평가 모델은 GPT-2 계열 모델(small-0.1B, medium-0.3B, large-0.7B)을 대상으로 수행하였다. 모델은 모두 Huggingface에서 제공되는 pretrained model을 이용했으며 모델 구조 또한 Huggingface transformer모델을 이용하였다.

Implementation. 양자화 과정의 구현 복잡도를 최소화하기 위해 abs-max 스케일링 방식을 적용하였으며, 스케일링 기법의 차이에 따른 영향을 본 연구의 주요 변수로 고려하지 않아 모든 모델에 동일한 스케일링 방식을 사용하였다. 양자화는 기존 Huggingface framework model이 사용하는 Conv1D 모듈을 Customized Conv1D로 대체해 MUXQ의 구조를 적용했다. Quantization이 적용된 GPT-2 레이어는 c_atttn, c_proj, c_fc, c_proj으로 총 네 레이어에 적용되었다. 그러나, 기존의 양자화 방식인 양자화 – 계산 – 복원 구조가 아닌, 양자화-복원-계산 구조를 이용하는 fake quantization[12]를 적용해 정확도를 측정했다. fake quantization의 정확도는 양자화 기법을 시뮬레이션한 결과와 비슷하지만, 추론속도는 시뮬레이션하지 못해 이번 연구는 추론 속도에 대한 실험적 비교는 진행하지 않았다. 또한 모든 실험은 NVIDIA GeForce RTX 3080 (12GB) 환경에서 수행되었다.

4.2 양자화 난이도

본 실험의 목적은 naive quantization과 MUXQ 간의 성능 차이가 두드러지게 나타나는 지점을 규명하는 것에 있다. 따라서 양자화 강도 조절 요소들을 적절히

표 1. GPT2 모델 규모 및 비트 정밀도에 따른 Naive Quantization, MUXQ, LLM.int8()의 perplexity비교.

model	granulrarity	layers	perplexity					
			IA(bits)	W(bits)	naive	MUXQ	llm.int8()	fp16
GPT2-small(0.1B)	per-vector	attention mlp	8	8	25.3825	25.28	25.2493	25.188
			7	8	25.8291	25.487	25.3867	25.188
			6	8	30.2001	26.9466	26.1048	25.188
			5	8	152.1251	43.129	30.5337	25.188
	per-tensor		8	8	50.618	**29.5534**	28.4595	25.188
GPT2-medium(0.3B)	per-tensor		8	8	22.5081	20.146	19.6749	18.4742
			7	8	35.8196	22.3645	20.4703	18.4742
			6	8	801.9368	38.2443	25.009	18.4742
GPT2-large(0.7B)			8	8	17.0662	16.7404	16.7041	16.4541
			7	8	19.4031	17.2334	17.0956	16.4541
			6	8	40.0405	19.2885	18.7935	16.4541

조합하여, MUXQ의 성능 향상이 실질적으로 발휘되는 조건을 분석하였다.

양자화 과정에서 조정 가능한 변수는 비트 정밀도(bit-precision), 스케일링 방법(scaling method), 세분화 수준(quantization granularity) 등이 존재한다. 본 실험에서 양자화의 강도를 조절하며 그 변화를 관측한다. 본 실험에서는 양자화 강도의 조절 방법으로는 다음 세가지 방법, 1) 활성값 정밀도 감소, 2) 가중치 정밀도 감소, 3) granularity 증가를 이용하였다. 이 세 방법을 통해 양자화의 난이도를 증가시킬 수 있었으며, 각 요소가 outlier handling에 어떤 영향을 미치는지 확인해볼 수 있었다. 이에 더해 model parameter의 크기에 따른 quantization 강도 변화도 함께 고려하였다.

4.3 실험결과

표1은 세 GPT2계열 모델의 wikitext-2 데이터셋을 통해 실험한 perplexity 결과를 나타낸다. 실험 결과, 대부분의 경우에서 MUXQ는 naïve quantization보다 perplexity가 LLM.int8()보다는 다소 떨어지는 모습을 보였다. 특히 본 실험에서 제시한 MUXQ의 outlier handling 성능수준은 1) 활성값 정밀도 감소를 통해 잘 확인해볼 수 있었다. 2) 가중치 정밀도 감소는 outlier handling method에 큰 영향을 미치지 않았으며, 3) granularity 변화를 통해서는 더 다양한 분석을 가능하게 해 MUXQ가 naive quantization보다 좋은 정확도를 보여주는 것을 더욱 확실하게 확인할 수 있다. 1) 활성값 정밀도 감소의 경우 본 실험에서 주요한 요소로 작용했다. 활성값의 outlier를 다루는 실험인 만큼 활성값 정밀도가 바뀜에 따라 MUXQ와 naive quantization과의 차이를 명확히 확인할 수 있었다. 표1를 보았을 때, 세 GPT2 모델 전부 특정 비트 이하로 활성값이 양자화되었을 때, naive quantization에서는 정확도가 치명적으로 낮아지고, MUXQ나 LLM.int8()의 경우는 그렇지 않은 모습을 확인해볼 수 있다. 특히 GPT2-medium의 경우 활성값 정밀도를 7-bit으로 낮추었을 때, naive quantization은 perplexity가 35 이상으로 크게 증가한 반면에, MUXQ와 LLM.int8()은 20~22수준에서 perplexity를 유지하는 모습을 보여주었다. 그러나, IA 정밀도를 6-bit수준으로 낮추었을 때에는, MUXQ의 perplexity는 LLM.int8()과 큰 차이가 나는 모습을 확인해볼 수 있다. 이는 활성값 양자화 난이도가 점점 올라갈수록, LLM.int8()의 FP16 outlier handling이 유리하게 작용하는 것을 의미한다.

표 2. GPT2-small, per-vector에서 가중치 정밀도를 다르게 했을 때 perplexity의 변화

perplexity					
IA(bits)	W(bits)	naive	MUXQ	llm.int8()	fp16
8	5	27.5149	27.3616	26.5359	25.188
8	4	41.642	41.1245	35.4289	25.188

표 2는 2) 가중치 정밀도 감소에 따른 실험결과는 세 방법 모두 비슷한 수준으로 성능 변화가 발생한 것을 나타낸다. 이는 MUXQ와 LLM.int8()모두 활성값 outlier handling을 주요하게 다루기 때문에 가중치 정밀도의 변화는 실험에서 사용된 세가지 방법에 큰 변수가 되지 못한 것으로 확인된다. 3) Granularity가 커질수록 기본적인 모델의 성능이 크게 증가했으며, 이에 따라 모델이 양자화에 더욱 강해짐(robust)을 확인할 수 있었다. 이를 이용해 활성값 정밀도 감소를 통해 원하는 결과를 명확하게 얻을 수 있었다. 6-bit 활성값 정밀도를 갖는 GPT2-large의 결과에서 MUXQ는 LLM.int8()과 비슷한 성능을 유지할 때, naive quantization는 perplexity값이 기존보다 2배이상 증가하며 크게 성능이 붕괴된 모습을 확인하였다.

4.4 토론

양자화 모델 추론(quantized model inference)의 목적은 기존 보다 크게 떨어지지 않는 선에서 양자화를 통해 보다 빠르게 추론하는 것이다. 본 실험에서는 MUXQ를 다른 양자화 기법들과 비교해 정확도가 떨어지지 않음을 확인했으나, 추론속도에 관해서는 실험을 모의하지 못했다. 하지만 matrix multiplication 연산에서 INT8의 연산속도가 FP16에 비해 속도면에서 장점을 가지기 때문에 양자화 오버헤드를 고려하더라도 속도우위를 예상해볼 수 있다.

V. 결론

본 연구에서는 대규모 언어모델의 저정밀 양자화에서 발생하는 활성값 outlier 문제를 근본적으로 완화하기 위해 MUXQ 를 제안하였다. 기존 방법들이 정확도 보존을 위해 mixed-precision 낮은 하드웨어 효율성을 갖거나 outlier channel 을 완전히 제거하지 못한 반면, MUXQ 는 INT 정밀도 내에서 outlier 의 magnitude 를 분산시켜 균형 잡힌 분포를 형성함으로써 정확도와 하드웨어 효율성을 동시에 확보하였다.

MUXQ 는 활성값 행렬에 보조 행렬을 추가하여 outlier 가 집중된 채널의 크기를 완화하고, 이를 통해 기존 naive quantization 보다 발전된 성능을 보여주었다. 또한 MUXQ 의 구조는 다른 양자화 방법과 충돌하지 않는 영역이기에 이에 추가적인 양자화 기법인 SmoothQuant, 혹은 다른 calibration

등의 방법을 적용할 수 있다는 점에서 발전 가능성이 존재한다.

결과적으로, MUXQ 는 소규모의 추가적인 연산을 실행하지만, 낮은 오버헤드를 유지하며, 균일한 INT 양자화 환경에서도 안정적인 LLM 추론 성능을 제공한다. 향후 연구에서는 MUXQ 를 다른 양자화 기법과 결합하거나, 실제 하드웨어 가속기의 속도·전력 효율 검증을 통해 실질적인 온디바이스 LLM 가속 방안으로 확장할 수 있을 것이다.

참고문헌

[1] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, "AI and memory wall," *IEEE Micro*, vol. 44, no. 3, pp. 33–39, 2024.

[2] M. Kim, S. Hong, R. Ko, S. Choi, H. Lee, J. Kim, *et al.*, "Oaken: Fast and efficient LLM serving with online-offline hybrid KV cache quantization," in *Proc. 52nd Annu. Int. Symp. Comput. Archit.* (ISCA), Jun. 2025, pp. 482–497.

[3] V. Sze, Y. H. Chen, T. J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.

[4] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.* (CVPR), 2018, pp. 2704–2713.

[5] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *Proc. Int. Conf. Mach. Learn.* (ICML), PMLR, Jul. 2023, pp. 38087–38099.

[6] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "GPT3.int8(): 8-bit matrix multiplication for transformers at scale," in *Advances Neural Inf. Process. Syst.* (NeurIPS), vol. 35, pp. 30318–30332, 2022.

[7] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.* (CVPR), 2018, pp. 2704–2713.

[8] S. Shen, Z. Dong, J. Ye, L. Ma, Z. Yao, A. Gholami, *et al.*, "Q-BERT: Hessian-based ultra-low-precision quantization of BERT," in *Proc. AAAI Conf. Artif. Intell.* (AAAI), vol. 34, no. 5, pp. 8815–8821, Apr. 2020.

[9] Y. Bondarenko, M. Nagel, and T. Blankevoort, "Understanding and overcoming the challenges of efficient transformer quantization," *arXiv preprint* arXiv:2109.12948, 2021.

[10] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, *et al.*, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Represent.* (ICLR), 2022.

[11] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," *arXiv preprint* arXiv:1609.07843, 2016.

[12] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," in *Low-Power Computer Vision*, Chapman and Hall/CRC, 2022, pp. 291–326.